

Enabling literal generation and/or link generation requires three additional pages of allocated memory. See Section 2.13 of this document for more information regarding memory allocation and related issues.

Using the /Q option switch in conjunction with the /O option switch will suppress error messages on generated links; however, the count of links generated is always included in the statistics page produced at the end of the assembly.

Assembly listing lines containing generated links will be flagged with **!**; this feature cannot be disabled.

Note: PAL8 and PAL10 are compatible with P?S/8 PAL with regard to all aspects of the disposition of generated link statements; however, all features regarding literals in PAL8 cannot be disabled. See below for detailed warnings regarding generated links that apply to all major PDP-8 assemblers.

2.07 Limitations of generated links

In general, generated links should be completely avoided; the short version: **Generated links do not work!**

While this simple caveat ought to be sufficient, a short study of the PDP-8 architecture reveals specific problems as follows:

- 1) Link generation is an attempt to partially abstract the PDP-8 architecture. Unfortunately, this leads to sloppy programming techniques that (might) function while violating the hard-wired rules of the PDP-8 instruction set.

A generated link is an indirect memory reference used to modify a malformed attempt at a direct memory reference that cannot be achieved; this is accomplished by using an address literal created in a manner analogous to that used within literal statements.

The program specifies a direct memory reference that is impossible due to the violation of memory addressing rules; instead an indirect reference is automatically created by the assembler (assuming this feature is enabled). Since this process has much in common with the creation of literals, all other aspects of literal generation applies.

The following assembled code fragment illustrates an attempt to violate the rules of the PDP-8 instruction set; the flawed statement is flagged as an error because link generation was not enabled.

```
          *0200          *200          /TYPICAL START ON PAGE 1.
IR 0577
000200 1377          TAD      577          /THIS REALLY CAN'T WORK.
```

The attempted usage cannot be achieved since the operand is outside of the allowed address range (0200-0377 in this instance); location 0577 is part of the following page (0400-0577). For an example such as this, locations on any page from 2 (0400-0577) through 37 (7600-7777) are equally inaccessible. The instruction is flagged with the **IR** error message (Illegal Reference).

The next example is the identical code fragment assembled with link generation enabled. There are no errors indicated, but the statement is flagged with **L** indicating link generation.

```

          *0200          *200          /TYPICAL START ON PAGE 1.
000200  1777'          TAD      577      /THIS REALLY CAN'T WORK.
          *0377
000377  0577

```

The generated code is an indirect instruction in the range of 1600-1777 as a consequence of the modification; a direct instruction would be in the range of 1200-1377. The code also creates an address constant; the table of literal/link addresses is dumped at the end of the assembly.

- 2) If link generation is enabled, the same source code produces flagged output. The instruction at 0200 is changed to TAD I 377. Additionally, location 0377 is generated containing 0577. Ignoring for the moment the instruction is now an indirect reference to 0577 using the pointer address 0377 as an intermediary, to a naive PDP-8 programmer (unaware of the potential negative consequences), the intended logic is satisfied: the contents of location 0577 are added to the accumulator.

Note: In these coding examples, symbolic references are deliberately not used to prevent clouding the issues. Actual attempts at link generation are usually in symbolic form; had the code been located in an accessible location, no link would be required. Links often come about when code overflows the intended page into the next page. By using purely numerical statements, the results are completely clear.

- 3) To the posited programmer, the program fragment works per se. Unfortunately, in real PDP-8 programming, considerations outside of this example tend to crop up; programs that had worked suddenly no longer work and might also cause unavoidable assembly errors of a kind that merely enabling link generation cannot remedy.

Such a programmer might be tempted to use a location already inaccessible (due to being on another page from the instruction) as a pointer. The assembler will properly mark this with the error message **II** (Illegal Indirect). Since it takes a pointer to access indirectly the address of the pointer, it is not possible to use it indirectly. Had the programmer obeyed the rules of the PDP-8 instruction set, this would have come about without sudden surprise.

Before investigating the insidious nature of link generation failing to work when no error messages occur (just generated link flagging), it will be shown that certain forms of generated links actually work as intended, as long as the mechanism is not pushed to the point of impossibility.

In all cases, links involving Direct JMP or Direct JMS instructions properly function in the sense described above. (Statements attempting link generation of the form JMS I or JMP I cannot function as explained above.) If this were the only issue, lazy programmers might have a valid excuse to enable link generation.

Unfortunately, all of the other forms of link generation will fail depending on the exact nature of the running program; the assembler cannot have knowledge of the programmer's intentions in the dynamic sense.

Repeating the generated link example from above:

```
          *0200          *200          /TYPICAL START ON PAGE 1.
000200  1777'          TAD      577      /THIS REALLY CAN'T WORK.
          *0377
000377  0577
```

A possible use of the example program includes manual start from the computer front panel; this allows setting the data field (DF) to a non-zero value in the range of 1 through 7.

When started this way, the program does not accomplish the intended function; the contents of location 0577 in the data field are added to the accumulator.

In general, PDP-8 programs are written for the 32K address space. The data field is often used as an internal convention of the program logic. When the program uses direct references (as the nominal statements attempt to do in the example), the data field plays no role in the logic of the program. Programmers who never enable link generation are certain their programs are designed to work as expected (as long as no other statements are flagged with errors).

Note: Lack of generated links is no guarantee there are no logic errors in the program; however, at least this area of bad design is not present.

- 4) As soon as extended memory factors creep into the logic of a program dependent on link generation, insidious failures start to occur. The programmer who never learned the discipline required to avoid this programming pitfall is now faced with a nearly insoluble dilemma. The only remedy to this situation is to bite the bullet so to speak, eliminating all generated links as a first step towards debugging the program; by preventing the attendant error messages much of the problem will be eliminated.

Note: Cleaning up after link generation might involve a large expenditure of effort all at once; had links never been used, the work necessary to eliminate assembly errors might have been accomplished in small increments.

In short, link generation correctly functions only if Direct JMP or Direct JMS instructions are involved. Link generation fails to *work* if any form of AND, TAD, ISZ or DCA instructions are involved. All forms of indirect usage attempts always fail.

Positing a stubborn and lazy PDP-8 programmer (the author has met people who fall into this category; their careers as PDP-8 programmers who use generated links are always short), one of those who insist on using link generation despite all the warnings; their excuses are invariably within the list as follows:

- a) Use only *JMP* and *JMS* links. (This is not to be believed.)
- b) Never use the cases that cannot work and count how many links are generated in the assembler statistics page every time to ensure nothing important has changed.

There are inherent logical fallacies in a) and b) above.

- c) One can only assume a) is true; it is necessary to check the entire program to ensure this is the case. Confirming this is far more work than merely avoiding link generation.
- d) The count of links generated may be the same as in a previous assembly. However, this is still no guarantee all of the generated links avoid the impossible cases; if the code is moved around sufficiently, a *JMP* or *JMS* link could drop out and be replaced with an impossible case in terms of the assembler generated links tally. It is still necessary to carefully check the entire program as in c) above; the unfortunate need for additional work still applies.
- e) In general, people this stubborn are hypocritical. They will tend to use the potentially impossible form of generated links claiming certain knowledge of the data field throughout the entire program. This makes them vulnerable to the various insidious problems described earlier.
- f) When cross-field calling conventions are established in meaningful programs, it is likely belatedly understood just how problematic the burden of link generation actually is. The work involved in elimination of the problems can be profound.
- g) In the case of P7S/8 PAL, PAL10 and PAL8, additional option switches must be set to suppress error messages associated with link generation. This is by design, as the developers of successful PDP-8 operating systems virtually never use link generation; they are trying to guide programmers to success in PDP-8 programming.